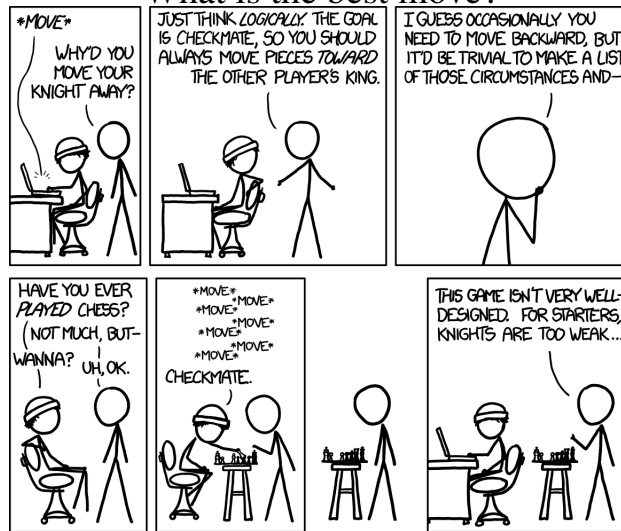
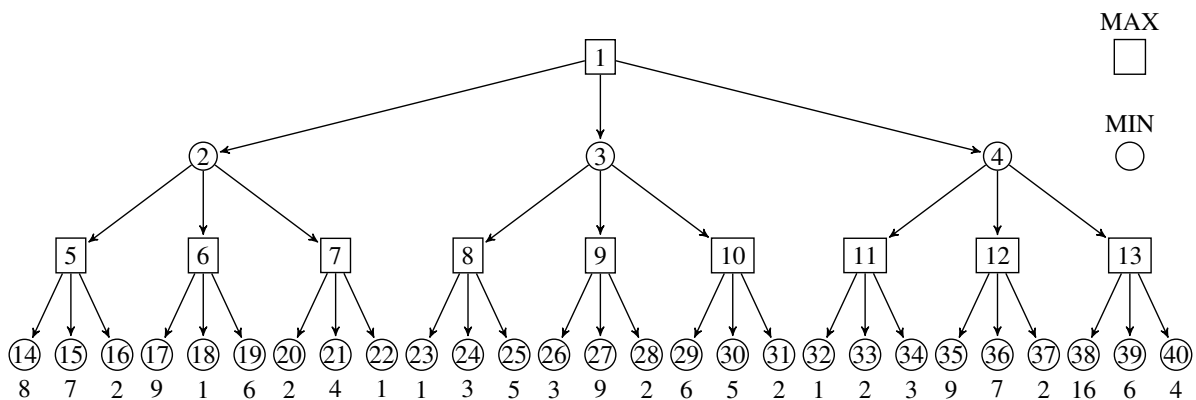


What is the best move?



PART 1: Minimax search

Consider the following game tree. Player MAX plays first and is represented with rectangles; MIN player is represented with circles. Numbers in each node are names used for convenience to refer to them (starting node is node 1). Finally, utility of leaf nodes are shown below them (e.g., the utility of node 21 is 4).



(a) Use minimax to determine the best move for MAX.

Hints:

We want you to come up with the solution. But we can give you some help:

- MAX moves to the node 3 with value of 5.
- Nodes 4 and 11 will have a utility of 3.
- Node 8 will have a utility of 5.

- (b) Which nodes will not be examined if the alpha-beta procedure is used?

Hints:

We want you to come up with the solution. But we can give you some help:

- There are 7 sub-trees pruned and not explored.
- There are two cuts on the sub-tree rooted at 2; three cuts on the sub-tree rooted at 3; and two cuts on the sub-tree rooted at 4.

Please give it a fair try with the above hints. If you still need additional guidance, please check this [video](#). However, note that if you needed to rely on the video, then there was probably insufficient learning. The best & expected case would be that you use the video to verify your approach. :-)

- (c) In which order will the nodes be examined by the alpha-beta procedure (assuming its depth-first implementation)?

Solution: (Note: this is a partial solution)

14, 15, 16, 5, 17, ..., 32, 33, 34, 11, 4

- (d) Did the alpha-beta procedure give the same best move as minimax?

Solution: Yes.

- (e) In general, does alpha-beta pruning change the result of mini-max? Prove your answer.

Solution: No. Alpha-beta pruning will never change the result, it will just make the process faster.

- (f) What are the properties of a “good” evaluation functions when using h-Minimax to avoid exploring the whole game tree?

Solution: The two primary qualities are *accuracy* and *speed*. As with heuristics, an evaluation function needs to be fast, because time spent generating values at terminal nodes is time that cannot be spent searching deeper in the search tree, as games are usually played with a fixed computational budget. If the evaluation function is super expensive, then it may be worth just exploring the sub-tree and hence having a guaranteed quality.

The evaluation function should also be as close to the real value returned by a minimax search as possible. Even more, it should be such that it preserves the rank ordering of states. So, for example, if the evaluation function happens to be the real utility plus a constant value for all nodes, then that would be a good function! Given that there is uncertainty due to computational limitations, for two-player zero sum games, this can be thought of as the probability of winning from any given state.

- (g) Are there any analogues to admissibility/consistency for heuristics that result in a provably optimal search? Why/why not?

Solution: This one is a bit trickier. Surely, if the evaluation function is *perfect* (i.e. it returns exactly the same value as a minimax search of the full game tree would), then it will clearly result in optimal choices. Optimal choices will also be obtained if the evaluation function is a linear function of the true utility value (i.e., $u(n) = a \times u(n) + b$, for any positive numbers $a, b \in \mathbb{R}^+$)

Beyond that, it is not clear what useful classes of function could be shown to guarantee optimality, in an analogous way as admissible heuristics for A*. The fact is that, in heuristic search, being optimistic

ensures optimality because one is forced to check every potential path which could be better than the current solution. In adversarial search, though, one is comparing two estimates against each other. Even if both are optimistic, which one is more optimistic may decide which action is chosen.

In theory if the evaluation function matches the true utility at terminal nodes; and its error can be found by a factor of ϵ in one backup step (i.e., guarantee that $|1 - v(n)/v(n')| < \epsilon$ where n' is a child of n), for non-terminal nodes, then given a finite game one could construct some bounds on the true values, and potentially use these in an alpha beta search. This would be hard to do in practice though.

- (h) Is minimax search guaranteed to find the best move against any possible opponent? If not, could it be that minimax on some opponent give worse outcome than minimax on a fully rational agent?

Solution: No, minimax may not give the best move if the opponent does not play fully rational (i.e., to its best) or there is an element of chance (e.g., dice): it may yield a suboptimal solution. In other words, in those cases, there may be a better move than what minimax will return. Remember, the key assumptions behind minimax is that the game is deterministic and that the opponent play optimally.

Nonetheless, we can guarantee that whatever minimax returns, the actual outcome will never be worse, regardless how the player plays. *Can you prove this?*

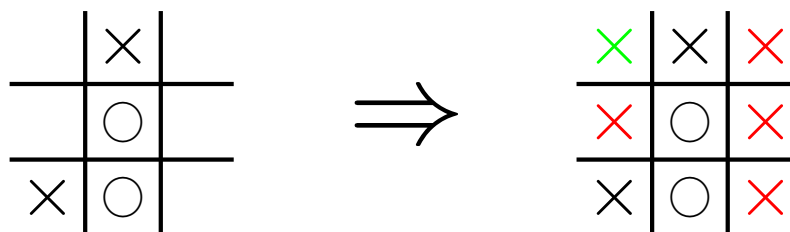
If there is some element of chance that is introduced (such as a die roll) or it is likely that an opponent will make a mistake in their move selection, then it is possible that minimax search will result in a sub-optimal result.

Which brings us to....

PART 2: Expectimax search

Minimax search relies on the assumption that both players (MIN and MAX) will play the game optimally. If this were the case then every game of noughts-and-crosses would result in a draw — however, anyone who has played against a small child knows that this is not the case (I hope you let them win!).

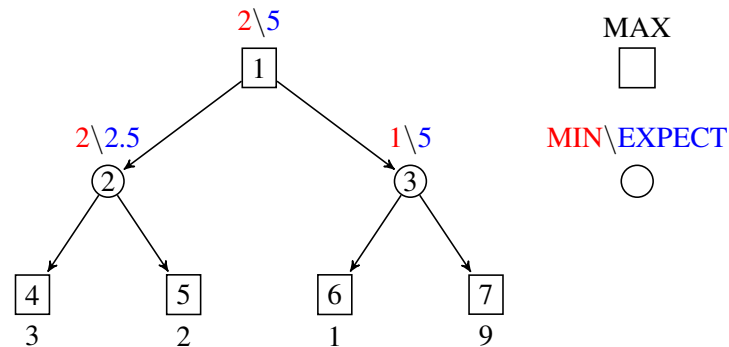
Take the following example of a game of noughts-and-crosses:



Here we have a situation on the left, where it is $\times$'s turn. If they put their cross in the top-left corner, then they are guaranteed to win in the next turn, however putting a cross in any other cell on the board is almost certainly going to result in a draw (provided $\bigcirc$ does nothing stupid). This is a reasonably trivial example, as anyone who has ever played the game before, can see that that this is the best move to make, but if it was the players first time playing — or the game was switched to something less trivial, like chess — the player might not see this as the optimal move, and put their cross somewhere else, allowing $\bigcirc$ to escape certain death.

It is in accounting for this uncertainty of play that is behind the idea of *expectimax* search. If minimax search is a model of the *worst-case* scenario for a given game, expectimax search is a model for the *average-case*. Here, instead of having MAX nodes and MIN nodes in the tree, we have MAX nodes and EXPECT nodes; with the EXPECT nodes taking the mean of the value of all of its children.

Consider the following example with minimax values in red and expectimax values in blue.



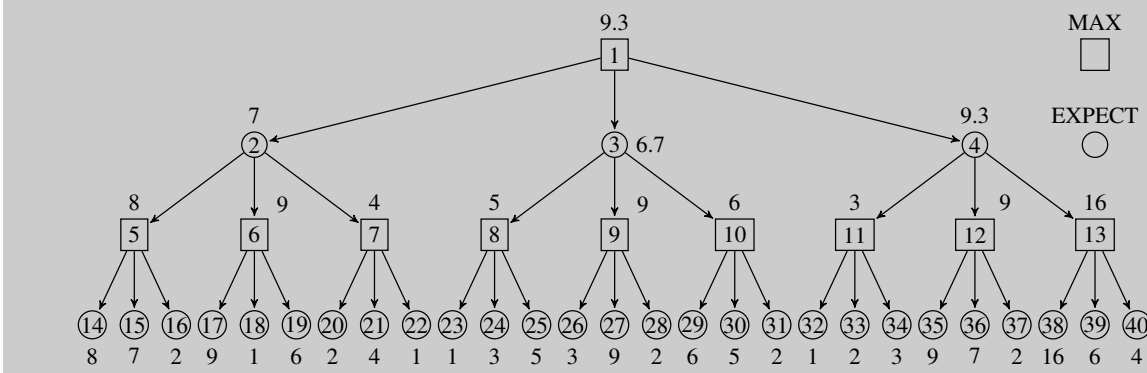
Here we can see that if minimax search is used then the left action will be chosen, with a utility of 2 and if expectimax is used, then the right action will be chosen with a utility of 5. Although it is possible that the opponent will choose the action that results in MAX obtaining a value of 1, the prospect of the possible 9 is enticing enough that it is worth the risk. It is important to note that there is no action that can be chosen that would ultimately result in a value of 5, this utility value is simply an average of the values that could be obtained if this action were chosen. The assumption here is that the opponent plays randomly; however, the actions can also be weighted to account for the fact that some choices are more likely than others. In the above example, when faced with two choices, one which will result in a utility of 1 for MAX and one which will result in a utility of 9 for MAX, even if the opponent is a very poor player, they will likely be able to distinguish between moves with such large differences in utility — and weighting these actions can account for this.

Another variant of this idea is *expectiminimax* search. Here, we have three types of nodes instead of two, MAX, MIN and EXPECT. Expectiminimax is used to model stochastic 2-player games, such as those where each player rolls a die and then makes a decision — the classic example used in the literature is backgammon. We will not go into detail about this technique, but it is mentioned here for the sake of differentiation.

- (a) Use expectimax search to determine the best move in the minimax game example from Part 1.

Solution:

MAX moves to node 4 with a value of 9.3.



- (b) Is the result the same as you obtained in Part 1?

Solution: No, expectimax moves to node 4 whereas minimax moves to node 3.

- (c) Can we use alpha-beta pruning with expectimax search?

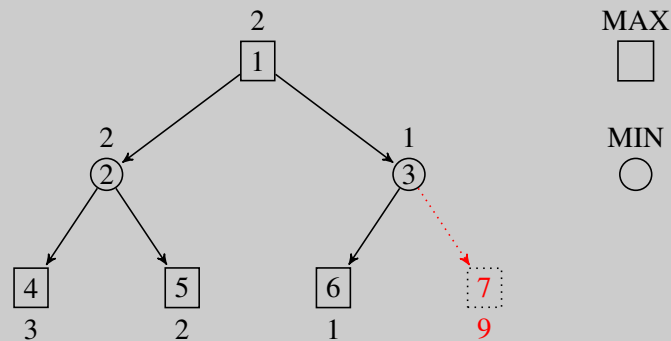
Solution:

The short answer is: No.

The slightly longer answer is: It depends.

The actual answer is: As expectimax search takes the average at its EXPECT nodes of the utility of all of its children, without looking at all of the children, there is no way to know what that average will be.

To use the example from above, if we are performing minimax search with alpha-beta pruning:



Once we have found the 1 in the second branch, we can immediately discount the rest of that branch, as there is no way that MIN will choose a value that is higher than 1. We cannot be so certain with expectimax search however. In this example, the EXPECT value for the left branch will be 2.5, so this means that if the bottom right value is at least 4, then the value at the EXPECT node for the right branch will be at least that of the left branch. Crucially, we cannot know this without first looking at the value, so there is no way that we can discount it a priori.

There is a circumstance where we *can* use alpha-beta pruning with expectimax search, and that is when the utility value for a state has an upper bound. Then we can keep a running average as we calculate the value of an EXPECT node, and if we end up in a situation where there is no way we can improve on a previous EXPECT node's value, even if the remaining children were this upper bound, then we can discard the branch.

End of tutorial worksheet